

多头借贷行为查询服务（dtjd）· API 接口文档与使用手册

面向接入方（客户）技术与运维人员的对外接口说明。

版本：dtjd（多头借贷行为） | 通信：HTTP + JSON | 编码：UTF-8

本服务采用统一请求信封（`appKey` / `sign` / `encryptionType` / `body`）与统一响应结构（`head` + `body`）。调用方对 `body` 业务参数按 MD5 规则加签即可。

本接口入参为个人三要素 `name`、`idCard`、`mobile`，三项均必填，缺任一项或格式非法将被直接拒绝且不计费。

查询结果为结构化对象，本服务将其序列化为 JSON 字符串置于 `body.result.range`，调用方需二次解析（见 3.1.5）。该对象包含决策类字段与数百个多头申请行为因子。

计费提示：本服务按有效查询次数计费——查得数据（001）与查无结果（999）均计费，详见第五章。

授权提示：调用前须取得被查询人的合法授权；授权书**不通过接口传输**，由调用方经商务渠道提交我方，详见 1.5。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「多头借贷行为查询服务（dtjd）」的第三方产品技术人员与日常运维人员。

1.2 接入须知

- 正式服务地址（任选其一，两域名等价）：`http://aiszcloud.cn:8080`、`http://aiszcloud.com.cn:8080`。
- 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。
- 本服务按有效查询次数计费（查得与查无均计费），请做好查询频次控制，避免短时间内对同一主体重复发起调用。
- 调用前须按 1.5 的要求准备并提交被查询人授权书。

1.3 接口说明

项目	说明
请求方式	POST（计费查询数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 ≥ 10 秒）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 <code>head.errorCode</code> / <code>body.code</code> 表达
签名	调用时需对 <code>body</code> 中的业务参数 + 我方分配的 <code>appSecret</code> 进行 MD5 加签，详见第二章

1.4 环境说明

- **正式环境**：使用正式账户，调用已开通接口，按有效查询次数计费（见第五章）。
- **测试环境**：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

1.5 授权要求

1. 调用本接口查询任何个人信息前，调用方**必须已取得被查询人合法、有效的授权**（个人信息查询授权书）。
2. 授权书**不通过 API 传输**：请求中无需、也不应携带任何授权文件、授权链接或授权类型字段，本接口的请求参数仅为 3.1.1 所列三要素。
3. 授权书由调用方**通过商务渠道提交给我方**，具体提交方式、文件格式与提交频次由商务在开通账户时与调用方约定；我方收到后进行审核、留存等后续处理。
4. 请在正式调用前与商务确认授权书提交方式并完成首次提交。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y890dtjd	String	是	我方分配给客户的公开标识
sign	7388c24a8546864b6c6589c86e8ee816	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

注意：本服务不需要额外的产品编号参数（产品由请求路径区分）。appKey / sign / encryptionType 不参与签名计算。
调用方传入的 body 字段一律为明文（姓名 / 身份证号 / 手机号）。

2.2 鉴权校验顺序

本服务按以下顺序校验，任一失败立即返回对应 head.errorCode（参数非法直接拒绝，不计费）：

- 1. appKey 是否存在 → 否则 505001
- 2. appKey 是否匹配到账户 → 否则 505004
- 3. 账户是否有效（启用且在有效期内）→ 否则 505007
- 4. 签名是否正确 → 否则 505002

2.3 加签方式

- 1. 取出 body 中所有非空的业务参数（不含文件/字节流类型，不含值为空的参数）。
- 2. 按参数名（key）的 ASCII 升序排序；首字符相同则依次比较后续字符。
- 3. 将排序后的参数按 参数名 参数值 直接拼接，最后追加 appSecret，得到待签名串。
- 4. 对待签名串做 MD5，取 32 位小写十六进制字符串，赋值给 sign。

示例：body = { "name": "张三", "idCard": "11010519491231002X", "mobile": "13809091009" } , appSecret = <你的密钥>。

按 ASCII 升序排序后 key 顺序为 idCard → mobile → name，待签名串为：

```
idCard11010519491231002Xmobile13809091009name张三<你的密钥>
```

sign = MD5(待签名串) 的小写十六进制值。

提示：拼接顺序由 key 的 ASCII 决定，请勿写死字段顺序。加签的范围是你实际发送的 body 全部非空字段——若额外传了本接口不需要的字段，它同样要参与加签，否则会被判 505002。

2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws Exception {
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()): # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest() # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 多头借贷行为查询 (dtjd)

项目	内容
路径	POST /v1/openapi/zlx/querySrmxDTJD
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/querySrmxDTJD (或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/querySrmxDTJD)
鉴权	appKey + MD5 签名 (见第二章)

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
name	张三	String	是	姓名, 明文传入
idCard	11010519491231002X	String	是	身份证号 (18 位, 末位可为 x), 须为校验位合法的有效号码, 明文传入
mobile	13809091009	String	是	手机号 (11 位, 以 1 开头), 明文传入

三项均必填。 任一字段缺失或格式非法 (身份证号非 18 位规范号码、手机号非 11 位) 将返回 `head.errorCode = 505062` , 直接拒绝, 不计费。
身份证号仅满足 18 位格式但校验位不正确时, 也可能返回 `505062` (服务处理异常), 请使用真实有效

的身份证号联调。

本接口查询须取得被查询人的合法授权。**授权书不在请求中携带**，由调用方通过商务渠道提交我方，我方进行后续处理（见 1.5）。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y890dtjd",
  "sign": "<按 2.3 规则计算>",
  "body": {
    "name": "张三",
    "idCard": "11010519491231002X",
    "mobile": "13809091009"
  }
}
```

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功（含查得/查无）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时需提供
time	189	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 查得数据【计费】；999 = 查无结果【计费】
msg	成功	String	业务描述
reqid	1kf9x2...	String	本次请求流水号
uid	lgt17901403102463	String	交易流水号（对账用）
result	{...}	Object	业务内容，仅 code = 001 时存在
result.range	"{...}"	String	多头借贷行为结果的 JSON 字符串（对象），需二次解析，结构见 3.1.5

3.1.4 响应示例

1) 查得数据（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 189,
"timestamp": 1718456789012 },
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "dkykagq5ml7u3",
    "uid": "lgt17901403102463",
    "result": {
      "range": "{\"Rule_final_decision\":\"Review\",\"Rule_final_weight\":\"20\",\"Rule_
name_QJF045\":\"在非银机构多次申请-低度\",\"Rule_weight_QJF045\":\"10\",\"als_m12_id_nban
k_allnum\":\"1\",\"als_m3_cell_nbank_orgnum\":\"1\"}"
    }
  }
}
```

（示例 range 仅节选部分字段，实际返回包含全部因子。）

2) 查无结果（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d5", "time": 96, "timestamp": 1718456789013 },
  "body": {
    "code": "999",
    "msg": "查无结果",
    "reqid": "lkf9x2ac",
    "uid": "lgt17901403109871"
  }
}
```

999 表示本次查询已完成、未查得该主体的相关记录——这是一个有效的查询结论，与 001 同样计费（见第五章）。

3) 网关级错误 (无 body)

```
{
  "head": { "errorCode": "505062", "errorMsg": "数据请求异常", "logId": "a1b2c3d6", "time": 12, "timestamp": 1718456789014 }
}
```

3.1.5 result.range 结构说明

result.range 是多头借贷行为结果序列化后的 JSON 字符串。调用方应先对该字符串做一次 JSON.parse / 反序列化，得到一个 JSON 对象（非数组）。

对象内的字段分为两类：**决策类字段**与**行为因子字段**。

一、决策类字段

字段	类型	说明
Rule_final_decision	String	最终决策结果：Accept = 通过；Reject = 拒绝；Review = 复议
Rule_final_weight	String	最终规则评分
Rule_name_XXXXXX	String	命中规则的名称，XXXXXX 为规则编号，如 Rule_name_QJF045 = 「在非银机构多次申请-低度」
Rule_weight_XXXXXX	String	对应规则的权重，与 Rule_name_XXXXXX 成对出现

规则类字段仅在命中时出现，未命中的规则不会下发对应键，请按「存在即解析、缺失即忽略」处理。

二、行为因子字段

因子字段数量较多（数百个），值均为 String 类型的数值文本。字段名按固定构词法拼装，可据此程序化解析：

```
als_<时间窗>_<查询主体>_<机构或业务类别>_<度量>
```

片段与取值	含义
时间窗: d7 / d15	近 7 天 / 近 15 天
时间窗: m1 / m3 / m6 / m12	近 1 / 3 / 6 / 12 个月
时间窗: fst / lst	距最早一次申请 / 距最近一次申请
查询主体: id / cell	按身份证号统计 / 按手机号统计
业务类别: pdl / caon / caoff / coon / cooff	线上小额现金贷 / 线上现金分期 / 线下现金分期 / 线上消费分期 / 线下消费分期
业务类别: rel / af / oth	信用卡 (类信用卡) / 汽车金融 / 其他
机构类别: bank (tra / ret)	银行机构 (传统银行 / 网络零售银行)
机构类别: nbank (p2p / mc / ca / cf / com / nsloan / autofin / sloan / cons / finlea / else)	非银机构 (P2P / 小贷 / 现金类分期 / 消费类分期 / 代偿类分期 / 持牌网络小贷 / 持牌汽车金融 / 持牌小贷 / 持牌消费金融 / 持牌融资租赁 / 其他)
时段: week / night	周末申请 / 夜间申请
度量: allnum / orgnum / selfnum	申请次数 / 申请机构数 / 在本机构申请次数
度量: tot_mons / avg_monnum / max_monnum / min_monnum	有申请记录月份数 / 平均每月申请次数 / 最大月申请次数 / 最小月申请次数
度量: max_inteday / min_inteday / inteday	最大申请间隔天数 / 最小申请间隔天数 / 间隔天数
度量: consnum / csinteday	连续申请次数 / 连续申请持续天数

示例解读:

字段名	含义
als_m12_id_nbank_allnum	按身份证号统计, 近 12 个月在非银机构的申请次数
als_m3_cell_nbank_orgnum	按手机号统计, 近 3 个月在非银机构的申请机构数
als_m12_id_nbank_finlea_orgnum	按身份证号统计, 近 12 个月在非银机构-持牌融资租赁机构的申请机构数
als_d7_cell_bank_night_allnum	按手机号统计, 近 7 天在银行机构夜间的申请次数
als_fst_id_nbank_inteday	按身份证号统计, 距最早一次在非银机构申请的间隔天数
als_lst_cell_nbank_consnum	按手机号统计, 最近开始在非银机构连续申请的次数

说明: result.range 的具体字段以实际返回为准, 本服务保证原样透出、不裁剪、不改写字段名。因子集合可能随数据丰富度变化, 建议调用方按「存在即解析、缺失即忽略」做容错处理, 不要假设某个因子键一定存在。

解析示例 (JavaScript) :

```
const data = JSON.parse(resp.body.result.range);

// 决策结果
if (data.Rule_final_decision === "Reject") {
  console.log("建议拒绝, 评分:", data.Rule_final_weight);
}

// 命中的规则 (成对出现)
Object.keys(data)
  .filter((k) => k.startsWith("Rule_name_"))
  .forEach((k) => {
    const no = k.replace("Rule_name_", "");
    console.log("命中规则:", data[k], "权重:", data["Rule_weight_" + no]);
  });

// 按需读取因子, 缺失即忽略
const n12 = data.als_m12_id_nbank_orignum;
if (n12 !== undefined) {
  console.log("近 12 个月非银申请机构数:", Number(n12));
}
```

3.2 计费查询数查询 (扩展接口)

查询本账户累计计费查询的次数与累计调用次数, 用于客户侧自助监控与对账。不返回额度上限/剩余量 (本服务无额度限制) 。

项目	内容
路径	GET /v1/openapi/zlx/quotaDTJD
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/quotaDTJD (或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/quotaDTJD)
鉴权	同 3.1 主查询 (请求体携带 appKey + sign 信封; body 可为 {}, 此时 sign = MD5(appSecret))

响应示例:

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1280
}
```

参数	说明
status	账户状态（ACTIVE / SUSPENDED 等）
serviceUsed	累计计费查询次数（code = 001 与 code = 999 均计入）
totalCalls	累计有效查询次数（含查得与查无；不含被网关拦截的请求）

说明：无任何额度上限拦截，仅做计费次数统计。本服务对查得与查无同等计费，故 serviceUsed 与 totalCalls 在正常情况下一致；对账请以 serviceUsed 为准。

3.3 健康检查

项目	内容
路径	GET /healthz
完整地址	http://aiszcloud.cn:8080/healthz （或 http://aiszcloud.com.cn:8080/healthz ）
鉴权	无
响应	HTTP 200，纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 body.code）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 服务处理异常 / 超时未决 / 系统错误（默认错误码）

服务侧的账户/参数/系统等异常均归一为网关 505062，不计费，并经异步复查兜底。

4.2 业务结果码 body.code（仅 errorCode = 0 时）

code	含义	是否计费
001	查得数据	计费
999	查无结果	计费

注：本服务按有效查询次数计费。只要本次查询产出了确定结论——无论是查得数据（001）还是确认无相关记录（999）——均计费一次。仅当返回 head.errorCode 非 0（未能产出结论）时不计费。

五、计费说明

body.code = 001（查得数据）与 body.code = 999（查无结果）均计入计费查询次数并对客户计费。本服务按有效查询次数计费：查无结果同样消耗一次查询并产出确定结论。网关级错误（head.errorCode 非 0：鉴权失败、参数非法、服务处理异常、系统异常等）一律不计费。计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。计费次数可随时通过计费查询数查询（3.2）接口自助核对。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 appKey、appSecret；服务地址见 1.2 接入须知。

2. 与商务确认被查询人授权书的提交方式，并按约定通过商务渠道提交授权书（见 1.5）；授权书不通过接口传输。
3. 按第二章实现加签，先在测试环境联调，再切正式环境。
4. 上线后通过计费查询数查询接口（3.2）监控调用量与计费次数。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（ ≥ 10 秒）。
- 收到网络超时/无响应时可安全重试：本服务基于内部流水号做幂等，不会因重试重复计费。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 505062、鉴权错误 505001 / 505002 / 505004），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 appKey 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex）；确认实际发送的每一个 body 字段都参与了加签
505007	联系商务确认账户状态与有效期
505062	检查 name、idCard、mobile 是否三项齐全，身份证号是否有效（含校验位）、手机号是否为 11 位；若入参正常仍持续出现，记录 logId 联系我方
code=999 持续出现	确认三要素是否为同一人的真实有效信息；该结果表示未查得相关记录，属有效结论，同样计费

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

6.4 联调自检清单

- ☐ 服务地址（`aiszcloud.cn:8080` 或 `aiszcloud.com.cn:8080`）、appKey、appSecret、环境匹配无误
- ☐ 已与商务确认授权书提交方式，并已通过商务渠道提交被查询人授权书（请求中不携带授权书）
- ☐ 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- ☐ name、idCard、mobile 三项均以明文传入且都不为空
- ☐ 使用校验位合法的真实身份证号（勿用随意编造的 18 位号码），手机号为 11 位真实号码
- ☐ 若额外传了本接口不需要的字段，已把它们计入加签范围
- ☐ 能正确解析 `head.errorCode` 与 `body.code` 两级状态
- ☐ 已知悉 999 查无结果同样计费，业务侧计费预估按有效查询次数而非查得数测算
- ☐ 已实现对 `result.range` 字符串的二次解析（结果为 JSON 对象）与字段容错
- ☐ 已按「存在即解析、缺失即忽略」处理决策规则字段与行为因子字段
- ☐ 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign，切勿泄露或随请求传输
logId	全链路追踪 ID (= head.logId) ， 排障/对账唯一凭据
uid	交易流水号 (= body.uid) ， 对账用
range	多头借贷行为结果的 JSON 字符串，需二次解析（结构见 3.1.5）
决策类字段	result.range 中以 Rule_ 开头的字段，给出最终决策与命中规则
行为因子	result.range 中以 als_ 开头的字段，按「时间窗 + 查询主体 + 机构/业务类别 + 度量」构词
有效查询	产出了确定结论的查询 (body.code = 001 或 999) ， 即本服务的计费单位
授权书	被查询人签署的个人信息查询授权文件；不通过接口传输，由调用方经商务渠道提交我方处理